Tradução PT-BR — Capítulo 6

Arquivo original: TG2_Bruno_Silveira.pdf

Capítulo traduzido: 6 Results (Resultados), Seções 6.1 a 6.4.

6 RESULTADOS

Este capítulo tem como objetivo apresentar e discutir os resultados de todos os experimentos propostos. A Seção 6.1 discute a compatibilidade da ferramenta com diferentes modelos, enquanto as Seções 6.2 e 6.3 comparam as medições reais de latência no MCU com métricas alternativas de estimativa de latência, respectivamente o número de MACs e o tempo de inferência na estação de trabalho. Por fim, a Seção 6.4 explora quanto tempo o profiling no dispositivo adicionou ao processo quando comparado a outros métodos.

6.1 Compatibilidade

Para avaliar a compatibilidade da ferramenta, registrou-se em quais modelos de teste todo o processo de profiling funcionou com sucesso e em quais ocorreram erros fatais. A Tabela 6.1 apresenta esses resultados.

Entre o total de 397 modelos testados, a taxa de sucesso foi de 97,48%. Possíveis causas esperadas de erro poderiam incluir ROM ou RAM insuficientes no MCU, incompatibilidade de operadores usados pelo TFLM, ou erros no dispositivo na conexão serial USB. Entretanto, todos os 10 modelos que falharam o fizeram pela mesma razão: um erro na ferramenta externa “stm32tflm”. Quando esses modelos são fornecidos como entrada para essa ferramenta de estimativa de tensor arena, ela imprime “HALTED” e fica travada indefinidamente. Observando os hiperparâmetros desses 10 modelos que falharam, mostrados na Tabela 6.2, não é claro o que fez com que a ferramenta STM32 rejeitasse esses modelos. Seus tamanhos em ROM estão na mesma faixa das variações bem-sucedidas, de modo que as que falharam não são nem menores nem substancialmente maiores que as demais.

6.2 Latência no MCU vs. MACs

Com o objetivo de avaliar a correlação entre o número de MACs e a latência medida no MCU, foram plotados os gráficos das Figuras 6.1, 6.2, 6.3 e 6.4. O desvio padrão dos 10 valores promediados para produzir cada ponto ficou na ordem de unidades, tão pequeno que não seria visível como barras de erro no gráfico; por isso, foi omitido. Como forma de quantificar a correlação, os coeficientes r para cada modelo são mostrados na Tabela 6.3.

Analisando os resultados, os quatro modelos base de benchmarking podem ser claramente separados em dois grupos: dois modelos com correlação quase perfeita (Classificação de Imagens e Detecção de Anomalias) e dois com correlação alta, porém não perfeita (Visual Wake Words e Keyword Spotting). Em termos de arquitetura, uma característica que se destaca nesses dois últimos é a presença de camadas Depthwise Convolution. O modelo de Detecção de Anomalias é o mais simples, contendo apenas camadas totalmente conectadas, o que possivelmente explica sua correlação quase perfeita. O que é um pouco surpreendente é o fato de o modelo de Classificação de Imagens também pertencer ao mesmo grupo, apesar de possuir blocos com camadas de convolução, pooling e conexões residuais.

Para ilustrar um cenário de pior caso para uma Busca de Arquiteturas Neurais usando MACs como proxy de estimativa de latência, destacam-se modelos específicos que exibiram a maior diferença proporcional entre a latência prevista e a latência real. Para variações de Keyword Spotting, considere o Modelo A (CHANNELS=64, KERNEL=6, FIRST_KERNEL=10×4), que possui 3,5M MACs e 147 ms de tempo de inferência no MCU, e o Modelo B (CHANNELS=128, KERNEL=3, FIRST_KERNEL=5×2), que possui 8,9M MACs e 151 ms de tempo de inferência no MCU. Olhando apenas para o número de MACs, estimaríamos que o Modelo B teria cerca de 2,5 vezes mais tempo de inferência do que o Modelo A e

possivelmente o descartaríamos na busca, quando na prática ele é apenas 1,02 vez mais lento. Um exemplo semelhante pode ser encontrado em variações de Visual Wake Words: comparando o Modelo C (INITIAL_CHANNELS=7, CHANNELS_MULTIPLIER=1.4, KERNEL=4) com 1,8M MACs e 131 ms de inferência, e o Modelo D (INITIAL_CHANNELS=9, CHANNELS_MULTIPLIER=2.0, KERNEL=3) com 4,1M MACs e 134 ms de inferência, esperaríamos que o Modelo D fosse 2,2 vezes mais lento do que o Modelo C, quando no MCU ele é de fato apenas 1,02 vez mais lento.

6.3 Latência no MCU vs. Latência na Estação de Trabalho

De forma análoga à seção anterior, os resultados do experimento comparando o tempo de inferência na estação de trabalho com o tempo de inferência no MCU estão nas Figuras 6.5, 6.6, 6.7 e 6.8. Neste caso, porém, os desvios padrão das 10 medições promediadas para os tempos de inferência na CPU da estação de trabalho foram significativos e, portanto, são representados com barras de erro no eixo x. Os coeficientes r para cada modelo podem ser vistos na Tabela 6.4.

Observando os coeficientes r para latência na estação de trabalho, percebe-se padrões diferentes em comparação ao número de MACs. Para esta métrica, apenas as variações de Classificação de Imagens apresentaram correlação quase perfeita, enquanto os modelos de Detecção de Anomalias, que anteriormente pertenciam ao mesmo grupo de Classificação de Imagens, agora apresentam correlação um pouco menor. Isso possivelmente pode ser atribuído à variância notavelmente alta que todas as variações de Detecção de Anomalias apresentam para a latência na estação de trabalho, visível na Figura 6.8. Os modelos de Visual Wake Words também têm coeficientes r muito altos — maiores do que ao usar MACs. Entretanto, o caso é diferente para Keyword Spotting. Para essas variações, a correlação é significativamente menor do que nas demais, e também menor do que ao usar MACs. Analisando o gráfico da Figura 6.7, há duas classes distintas de variações, cada uma caindo em uma das linhas de correlação. Essa separação não é vista no gráfico para MACs (Figura 6.3), indicando que ela reflete algum gargalo particular da configuração específica da estação de trabalho.

Como Keyword Spotting apresenta a menor correlação entre latência na estação de trabalho e latência no MCU, este é o foco para o cenário de pior caso em NAS. Um processo de busca guiado por latência na estação de trabalho estimaria o Modelo E (CHANNELS=96, KERNEL=6, FIRST_KERNEL=20×8), com 1132 µs de latência na estação de trabalho, como 5,41 vezes mais lento do que o Modelo F (CHANNELS=128, KERNEL=5, FIRST_KERNEL=5×2), que teve 209 µs. Essa escolha não refletiria os tempos reais no MCU: o Modelo E (258K µs no MCU) é, na realidade, 1,01 vez mais rápido que o Modelo F (262K µs no MCU), em vez de 5,41 vezes mais lento.

6.4 Tempo de Execução

O tempo de execução de cada etapa da ferramenta B.A.L.A.S foi medido, e os tempos médios para perfilar uma variação de modelo são exibidos na Figura 6.9. Algumas etapas, como arena_time, mac_time e cpp_codegen, tiveram tempos de execução desprezíveis — na ordem de unidades de milissegundos — e, portanto, optou-se por não mostrá-las no gráfico, pois seriam invisíveis e seus rótulos ocupariam espaço, desviando atenção das etapas mais relevantes. Usando o número de tentativas necessárias, foi possível deduzir quanto tempo foi gasto aguardando timeout, recompilando e redeployando; essas barras em vermelho representam o tempo desperdiçado por subestimar o tamanho da tensor arena.

A partir da Figura 6.9, observa-se claramente como características únicas de cada modelo base refletem nos tempos de execução. O modelo Visual Wake Words possui, de longe, o maior tamanho de entrada (Tabela 4.1), o que explica por que a etapa de transferência de dados domina o tempo total. A

transferência de dados leva mais tempo do que a inferência para todos os modelos, exceto Keyword Spotting, que, em contraste com Visual Wake Words, tem o menor tamanho de entrada. Considerando que o desvio padrão do tempo de inferência no MCU foi desprezível em todos os modelos e variações, uma otimização óbvia seria executar apenas uma inferência em vez de promediar 10. Isso reduziria o tempo da etapa de inferência em 10 vezes e, mais importante, também reduziria a etapa de transferência de dados na mesma proporção.

Outro aspecto importante visível na Figura 6.9 é o quanto as re-tentativas causadas por subestimativas de tamanho de tensor arena afetaram o tempo total. Para os dois modelos em que isso ocorreu (Keyword Spotting e Detecção de Anomalias), o tempo médio por variação profilada aumentou em 1,8 e 2,4 vezes, respectivamente. Nenhuma variação testada precisou de mais de 3 tentativas para atingir o tamanho requerido da tensor arena, o que significa que, se uma constante de 10 kB tivesse sido somada à estimativa do stm32tflm para compensar subestimativas, todo o tempo desperdiçado em re-tentativas teria sido eliminado. Contudo, isso não garante eliminação de re-tentativas para qualquer modelo, apenas para as variações testadas neste trabalho.

Considerando as otimizações propostas, seria possível ter 10 vezes menos tempo de execução para inferência e transferência de dados e eliminar o tempo desperdiçado em re-tentativas. A Figura 6.10 ilustra como os resultados de tempo seriam com essas otimizações. Os experimentos não foram executados uma segunda vez com as otimizações implementadas; em vez disso, foi apenas calculado como os resultados seriam. Isso mostra que é possível perfilar tempos de inferência de modelos reais de TinyML em hardware real de MCU com média em torno de 20 segundos por modelo.